

第五章 轮式机器人各模块设计与编程实验

该实验共有四个模块实验，分别是直流电机转动测试、编码器数据读取、线性 CCD 数据的读取。拓展实验内容：使用 Arduino 读取超声波测距传感器数据。

实验模块一 直流电机转动测试实验

一、实验目的

- 1、了解电机的结构、组成及控制原理
- 2、了解电机控制的要点及控制方法

二、实验器材

Arduino 开发板，电机驱动模块 L298N 或自制驱动模块，锂电池，直流电机，电源模块，面包板，杜邦线。

三、实验原理与实验方法

1、接线

如图 5.1 所示，直流电机的控制很简单，在电机的正负端（即图中 1 和 6 端口）接上一定的电压就可以运转。图 5.1 另外四根线是用于测量电机转速的编码器接口。提到控制电机，则有两个关键因素：一是速度，二是方向。本实验选用的电机工作电压 7~13V，电流 540mA。一般选择单片机控制器来控制电机，Arduino 的电压为 5V，I/O 口输出的电流最大为 40mA，不足以驱动电机运转，因此需要一个中间介质，那就是驱动电路，保证给电机提供一定功率的输出。图 5.2 左图给出驱动直流电机的一种驱动模块 L298N，它的内部结构是由 MOS 管搭建的 H 桥电路。

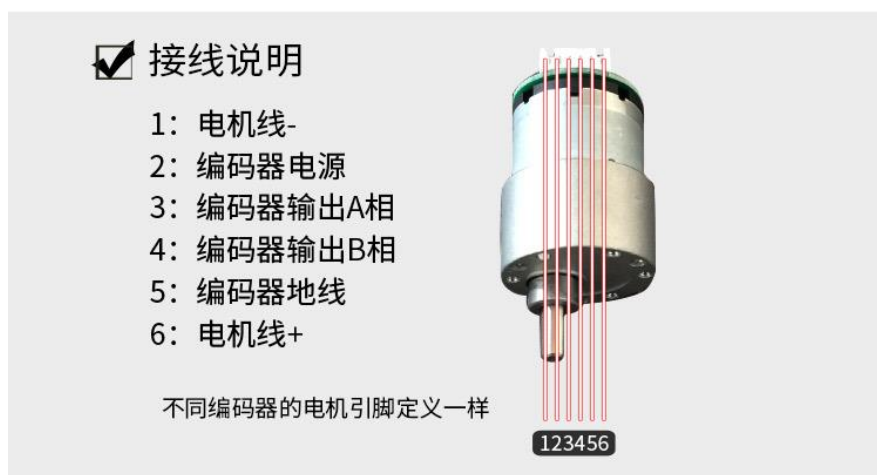


图 5.1 电机与驱动模块接线

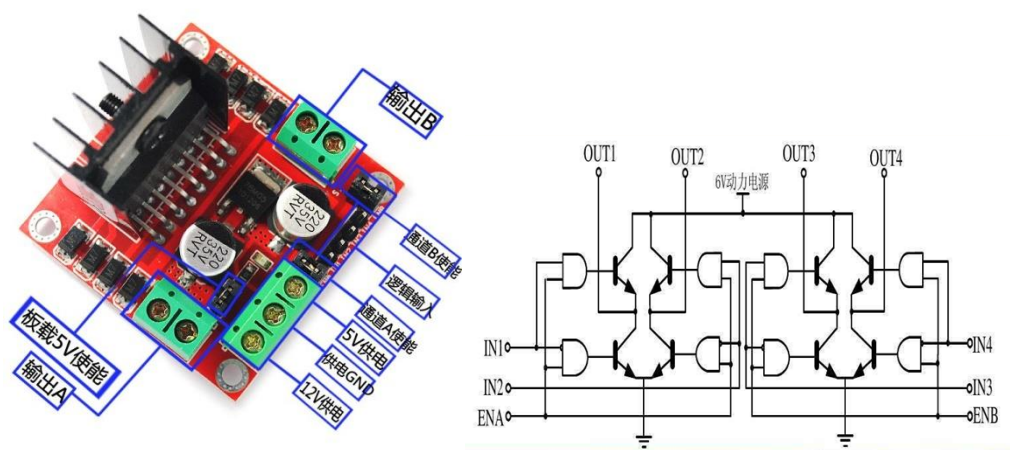


图 5.2 L298N 模块结构图与内部原理图

由于 L298N 是 2 路的 H 桥驱动，所以可以同时驱动两个电机，分别从调速端 A（即通道 A 使能 ENA）输入 PWM 信号，IN1 和 IN2 输入高低电平驱动电机 1 的转速和方向，分别从调速端 B（即通道 B 使能 ENB）输入 PWM 信号，IN3 和 IN4 输入高低电平驱动电机 2 的转速和方向。L298N 驱动模块原理图和逻辑时序图分别如图 5.2 和 5.3 所示。

Arduino+电机+驱动模块 L298N 参考接线：（非常重要，线路接法参照下面）

- （1）Arduino 数字引脚 2→电机 3 引脚编码器输出 A 相
- （2）Arduino 数字引脚 3→电机 4 引脚编码器输出 B 相
- （3）Arduino 数字引脚 10→驱动器 L298N ENA
- （4）Arduino 数字引脚 11→驱动器 L298N IN2
- （5）Arduino 数字引脚 12→驱动器 L298N IN1
- （6）驱动器 L298N 输出 A01→电机 6 引脚电机+（或电机 1 引脚电机-）
- （7）驱动器 L298N 输出 A02→电机 1 引脚电机-（或电机 6 引脚电机+）
- （8）L298N 模块：12V 供电接 12V 电源；5V 供电接到 5V 电源，GND 接电源的 GND，电源的 GND 接 Arduino 的 GND。
- （9）电机上 2 引脚接 5V，5 引脚接 Arduino GND。

注意：绝对不可以把电池 12V 电压直接接到 Arduino Uno 板卡任何一个引脚上。

注意：绝对不可以把电池 12V 电压直接接到 Arduino Uno 板卡任何一个引脚上。

注意：绝对不可以把电池 12V 电压直接接到 Arduino Uno 板卡任何一个引脚上。

直流电机	旋转方式	IN1	IN2	IN3	IN4	调速PWM信号	
						调速端A	调速端B
M1	正转	高	低	/	/	高	/
	反转	低	高	/	/	高	/
	停止	低	低	/	/	高	/
M2	正转	/	/	高	低	/	高
	反转	/	/	低	高	/	高
	停止	/	/	低	低	/	高

图 5.3 L298N 模块逻辑时序图

2、PWM 调速

PWM 调速是通过改变输出方波的占空比使负载上的平均电流功率从 0~100%变化，从而改变电机速度。利用 PWM 调制的优点是电源的能量功率能得到充分的利用、电路的效率最高。例如：当输出为 50%的方波时，PWM 电路输出能量功率也为 50%，即几乎所有的能量都转换给负载。而采用常见的电阻降压调速时，要使负载获得电源最大 50%的功率，电源必须提供 71%以上的输出功率，这其中 21%消耗在电阻的压降及热耗上。

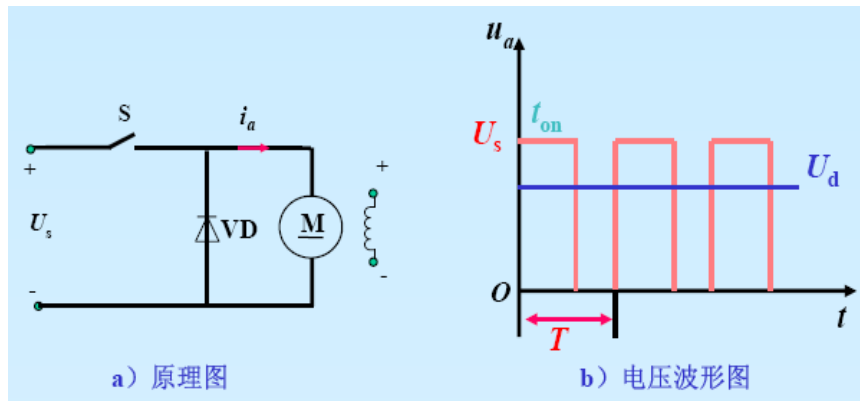


图 5.4 直流斩波器——电动机系统原理图和电压波形

在图 5.4 中，VT 表示电力电子开关器件，VD 表示续流二极管。当 VT 导通时，直流电源电压 U_s 加到电动机上；当 VT 关断时，直流电源与电机脱开，电动机电枢电流经 VD 续流，两端电压接近于零。如此反复，电枢端电压波形如图，好像是电源电压 U_s 在 t_{on} 时间内被接上，又在 $T - t_{on}$ 时间内被斩断，故称“斩波”。电动机得到的平均电压为：

$$U_d = \frac{t_{on}}{T} U_s = \rho U_s$$

式中：

T - 电力电子开关器件的开关周期；

t_{on} - 开通时间；

ρ - 占空比， $\rho = \frac{t_{on}}{T} = t_{on}f$ 其中： f - 开关频率。

3、单个电机全速转动测试（参考程序）

```
1. #define PIN_EN 10//定义 PWM 输出引脚
2. #define PIN_AIN2 11//定义正反转控制引脚
3. #define PIN_AIN1 12//定义正反转控制引脚
4. void setup() {
5.     pinMode(PIN_EN, OUTPUT); //定义引脚为输出
6.     pinMode(PIN_AIN2, OUTPUT);
7.     pinMode(PIN_AIN1, OUTPUT);
8. }
9. void loop() {
10.    digitalWrite(PIN_EN, HIGH);
11.    digitalWrite(PIN_AIN2, HIGH);
12.    digitalWrite(PIN_AIN1, LOW);
13. }
```

4、使用 PWM 波调整电机转速

```
1. #define PIN_PWM 10//定义 PWM 输出引脚
2. #define PIN_AIN2 11//定义正反转控制引脚
3. #define PIN_AIN1 12//定义正反转控制引脚
4. void setup() {
5.     pinMode(PIN_PWM, OUTPUT); //定义引脚为输出
6.     pinMode(PIN_AIN2, OUTPUT);
7.     pinMode(PIN_AIN1, OUTPUT);
8. }
9. void loop() {
10.    analogWrite(PIN_PWM, 100);
11.    digitalWrite(PIN_AIN2, HIGH);
12.    digitalWrite(PIN_AIN1, LOW);
13.    delay(3000);
14.
15.    analogWrite(PIN_PWM, 100);
16.    digitalWrite(PIN_AIN2, LOW);
17.    digitalWrite(PIN_AIN1, HIGH);
18.    delay(3000);
19.
20.    analogWrite(PIN_PWM, 200);
21.    digitalWrite(PIN_AIN2, HIGH);
22.    digitalWrite(PIN_AIN1, LOW);
23.    delay(3000);
24.
25.    analogWrite(PIN_PWM, 200);
26.    digitalWrite(PIN_AIN2, LOW);
```

```
27.  digitalWrite(PIN_AIN1, HIGH);  
28.  delay(3000);  
29. }
```

四、思考问题

- 1、直流电机调速方法。
- 2、两个直流电机的同步控制方法。

实验模块二 编码器数据读取

一、实验目的

了解编码器原理

二、实验设备

Arduino 控制板，电机，锂电池，电源模块，面包板，杜邦线。

三、实验原理

M 法是测量单位时间内的脉数换算成频率，因存在测量时间内首尾的半个脉冲问题，可能会有 2 个脉冲的误差。速度较低时，因测量时间内的脉冲数变少，误差所占的比例会变大，所以 M 法宜测量高速。如要降低测量的速度下限，可以提高编码器线数或加大测量的单位时间，使一次采集的脉冲数尽可能多。

使用 M 法测速的时候，会通过测量单位时间内 A 相（或 B 相）的上升沿或者下降沿，也就是图 5.5 中对应的数字 1234 中的某一个，这样图中三个脉冲就只能计数 3 次。四倍频的方法是测量 A 相和 B 相编码器的上升沿和下降沿，这样在同样的时间内，可以计数 12 次（3 个 1234 的循环）。（相当于对编码器的细分）

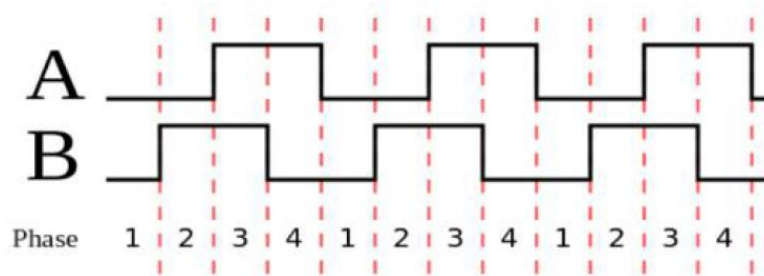


图 5.5 编码器输出波形图

因为编码器输出的是标准的方波，所以可以使用 Arduino UNO 等单片机工具直接读取。在软件中的处理方法分两种，自带双路编码器接口的单片机如 STM32，可以直接使用硬件计数。而没有编码器接口的单片机如 Arduino UNO，可以通过外部中断读取，比如把编码器 A 相输出接到单片机的外部中断入口，这样就可以通过跳变沿触发中断，然后在对应的外部中断服务函数里面，通过 B 相的电平来确定正反转。如当 A 相来一个跳变沿的时候，如果 B 相是高电平就认为是正转，低电平就认为是反转。

本实验采取边沿触发计数方式，编码器 A、B 相只要有跳变，就计数；将电机控制的周期定位 20ms 一个周期，那么同时编码器计数也是每 20ms 为一个计数周期，之后清零，再重新计数。以下给出计数步骤和计数值与电机转速的计算方法：

- 1、边沿触发计数
- 2、20ms 定时器计时计数
- 3、设 20ms 编码器计数 X 个脉冲，则转速为：

$X/20\text{ms}=X*50/\text{s}=X*5 \text{ 转/s}$ ，这里假设电机每转一圈，编码器发出 10 个脉冲。

注：本章节暂时不使用电机转速计算，后面实验第六章节会介绍。

四、编码器数据读取参考程序

```
1. #define ENCODER_A 2 //编码器读取采用中断方式，该引脚为第 0 个中断接口
2. #define ENCODER_B 3
3. long encoderVal; //左编码器值
4. //函数功能：外部中断读取编码器数据，具有二倍频功能注意外部中断是跳变沿触发
5. void getEncoder(void) {
6.     if (digitalRead(ENCODER_A) == LOW) { //如果是下降沿触发的中断
7.         if (digitalRead(ENCODER_B) == LOW) {
8.             encoderVal--; //根据另外一相电平判定方向
9.         }
10.        else {
11.            encoderVal++;
12.        }
13.    }
14.    else { //如果是上升沿触发的中断
15.        if (digitalRead(ENCODER_B) == LOW) {
16.            encoderVal++; //根据另外一相电平判定方向
17.        }
18.        else {
19.            encoderVal--;
20.        }
21.    }
22. }
23. void setup() {
24.     //编码器引脚都设置输入
25.     pinMode(ENCODER_A, INPUT);
26.     pinMode(ENCODER_B, OUTPUT); //此处有错误，请找出并纠正
27.     //下面这一条语句此处有错误，请找出并纠正
28.     attachInterrupt(2, getEncoder, CHANGE); //使能编码器引脚外部中断 0，对应 Arduino 引脚
29.     Serial.begin(9600);
30. }
31. void loop() {
32.     delay(100);
33.     Serial.print("encoder val = ");
34.     Serial.print(encoderVal);
35.     Serial.print("\r\n");
36. }
```

注意：上面程序中有两处错误，请找出来并更正，程序才能正常运行。写入程序后，

转动你的电机轮子，看看串口观察器里的数值变化。

五、思考问题

- 1、如何通过编码器获得轮子的转速。（提示:我们所使用的电机输出轴，即连轮子的轴每转一圈，编码器输出 390 个脉冲）
- 2、编码器测速方法。

拓展实验：使用 Arduino 读取超声波测距

一、实验目的

掌握 Arduino 读取各类传感器数据的方法；

掌握传感器数据处理的方法。

二、实验设备和材料

Arduino UNO，超声波测距传感器，8 路寻迹传感器，面包板，杜邦线。

三、传感器原理及使用方法

1、超声波传感器

超声波传感器主要技术参数：

使用电压：DC5V；

静态电流：小于 2mA；

电平输出：高 5V；

电平输出：低 0V；

感应角度：不大于 15 度；

探测距离：50px-11250px，高精度可达 5px；

接线方式端口：VCC（电源）、trig（控制端）、echo（接收端）、GND（地）。



图 5.8 超声波测距传感器实物图

超声波测距传感器工作原理：

(1) 采用 I/O 触发测距，给至少 10us 的高电平信号；

(2) 模块自动发送 8 个 40kHz 的方波，自动检测是否有信号返回；

(3) 有信号返回，通过 I/O 输出一高电平，高电平持续的时间就是超声波从发射到返回的时间。

测试距离=(高电平时间*声速(340M/S))/2。

超声波传感器主要利用声波传播时间测距原理，通过晶振向外发射超过人体能感知的高频超声波；一般选用 25~40kHz 波，然后控制模块检测反射回来波，模块计算声波传输时间来进行测距。

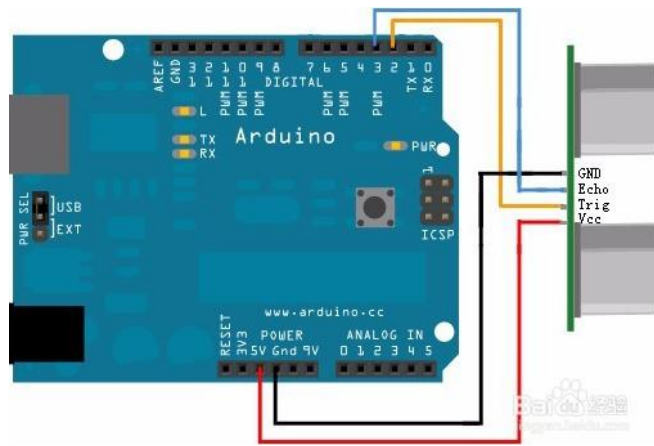


图 5.9 Arduino 与超声波测距传感器接线

参考程序：

```

1.  const int TrigPin = 2;
2.  const int EchoPin = 3;
3.  float cm;
4.  void setup()
5.  {
6.    Serial.begin(9600);
7.    pinMode(TrigPin, OUTPUT);
8.    pinMode(EchoPin, INPUT);
9.  }
10. void loop()
11. {
12.   digitalWrite(TrigPin, LOW); //低高低电平发一个短时间脉冲去 TrigPin
13.   delayMicroseconds(2);
14.   digitalWrite(TrigPin, HIGH);
15.   delayMicroseconds(10);
16.   digitalWrite(TrigPin, LOW);
17.   cm = pulseIn(EchoPin, HIGH) / 58; //将回波时间换算成 cm
18.   cm = (int(cm * 100)) / 100; //保留两位小数
19.   Serial.print(cm);
20.   Serial.print("cm");
21.   Serial.println();
22.   delay(1000);
23. }

```

第六章 直流电机调速实验

直流电机调速实验分为三种方案（任选其一做）：基于 L298N 驱动模块的直流电机调速、基于自己焊接的电机驱动板的直流电机调速和基于 TB6612 驱动模块的电机调速。

一、实验目的

- 1、掌握直流电机调速的方法；
- 2、理解 PID 调速原理及方法。

二、实验器材

Arduino，直流电机（轮式小车车体），驱动模块，电源模块，锂电池等。

三、实验方法与步骤

- 1、电机与驱动模块接线；
- 2、编码器测速，通过对编码器 A、B 相的计数来计算电机的速度，转速以弧度/s 计算；（参考编码器读取实验，在中断里面使用的全局变量，需要定义成 **volatile** 类型，例如：**volatile long encoderVal;**）

采取边沿触发计数方式，编码器 A、B 相只要有跳变，就计数；将电机控制的周期定时 PERIOD 为一个周期，那么同时编码器计数也是每 PERIOD ms 为一个计数周期，之后清零，再重新计数。以下给出计数步骤和计数值与电机转速的计算方法：

- 1) 边沿触发计数
- 2) 定时器定时计数

3) 大家的电机是轮子每转动一圈输出 390 个脉冲，实验三编码器读取使用边缘触发中断，则可以实现脉冲的上升和下降计数。所以在计算速度的时候应采用 $390 \times 2 = 780$ 来计算速度。以下 PERIOD 为测速周期，ms 为单位，encoderVal 为记录到的边缘触发次数（为脉冲数的 2 倍），则速度为

$$\text{velocity} = (\text{encoderVal} / 780.0) * 3.1415 * 2.0 * (1000 / \text{PERIOD});$$

注意：可以使用 **millis()** 或者 **MsTimer2** 来实现周期定时，建议使用 **MsTimer2** 来实现定时，**millis()** 精度较差。**MsTimer2** 的使用，需要通过 **Arduino IDE** 的库管理进行安装（参考前面温湿度传感器的库安装方法）。以下定时器代码供参考：

```
1. #include <MsTimer2.h>
2. //定时器库的头文件，除了使用 MsTimer2，也可以使用 millis(),但很容易受到干扰，定时不准
3. void control()
4. {
5.     //这里面可以放置编码器测速算法和电机转速 PID 控制算法
6. }
7. void setup()
```

```

8. {
9.     MsTimer2::set(PERIOD, control); //设置每隔 PERIOD 时间，执行 control 函数一次
10.    MsTimer2::start();
11. }
12. void loop()
13. {
14. }

```

3、P（比例）或 PI（比例积分）控制器算法设计；

增量式 PID 算法，由以下公式计算获得 PID 控制器的输出值：

$$\Delta u(k) = u(k) - u(k-1) = q_0 e(k) + q_1 e(k-1) + q_2 e(k-2)$$

其中 $\Delta u(k)$ 为增量式 PID 的输出， $e(k)$ ， $e(k-1)$ ， $e(k-2)$ 分别为当前时刻偏差，前一时时刻偏差，更前一时时刻偏差， K_p 为比例增益， T_i 为积分时间， T_d 为微分时间。

$$q_0 = K_p \left(1 + \frac{T}{T_i} + \frac{T_d}{T}\right)$$

$$q_1 = -K_p \left(1 + \frac{2T_d}{T}\right)$$

$$q_2 = K_p \frac{T_d}{T}$$

或者写作：

$$\Delta u(k) = K_p \{e(k) - e(k-1)\} + K_i e(k) + K_d \{e(k) - 2e(k-1) + e(k-2)\}$$

其中：

$e(k)$ 为当前时刻偏差	K_p 为比例系数
$e(k-1)$ 为前一时时刻偏差	K_i 为积分系数
$e(k-2)$ 为更前一时时刻偏差	K_d 为微分系数

注意：大家可以把上述算法编写成一个 **pidController** 的函数，如

```
int pidController(float targetVelocity, float currentVelocity)
```

```
{
```

```
    //targetVelocity 为目标速度， currentVelocity 为当前速度
```

```
    //这里实现算法，注意上述算法求出来的是  $\Delta u(k)$ ，所以要和前一次//  

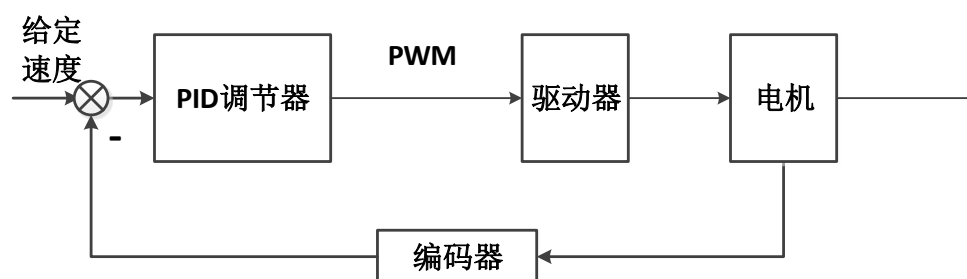
    的输出 u 相加才能得到当前输出值。所以需要使用全局变量保存上//  

    一时刻  $u(k)$ ，上一时刻的误差  $e(k-1)$ ，上上时刻的误差  $e(k-2)$ 。
```

```
    return 当前时刻计算得到的控制输出；
```

```
}
```

4、将系统构成闭环，给定电机或小车初始速度，让其运动；



5、通过调节 K_p （比例）、 T_i （积分时间）和 T_d （微分时间）参数，使电机或小车达到给定速度控制（使用课上讲的秘诀）；

PID 参数调节过程比较繁琐、枯燥，请同学结合课堂理论知识进行调节和尝试。在实际工程中，PID 参数调节需要丰富的实际经验，以下工程中总结出来的 PID 参数调试口诀，供各位学生参考：

参数整定找最佳，从小到大顺序查；
 先是比例后积分，最后再把微分加；
 曲线振荡很频繁，比例度盘要放大；
 曲线漂浮绕大湾，比例度盘往小扳；
 曲线偏离回复慢，积分时间往下降；
 曲线波动周期长，积分时间再加长；
 曲线振荡频率快，先把微分降下来；
 动差大来波动慢，微分时间应加长；
 理想曲线两个波，前高后低 4 比 1；
 一看二调多分析，调节质量不会低。

四．实验报告

- (1) 根据实验不同参数的数据，画出闭环控制特性曲线。
- (2) 总结闭环系统的 PID 参数调节心得。

附录：程序框架供参考（自己编写程序的同学，可以以此为框架编写程序）

```
1. #include <MsTimer2.h>           //定时器库的头文件
2. //定义电机编码器 A B 相引脚（自行补充引脚号）
3. #define ENCODER_A                //注意这里需要补充接入中断引脚
4. #define ENCODER_B
5. //定义电机驱动的控制信号（自行补充引脚号）
6. #define PWML
7. #define INL1
8. #define INL2
9. //全局变量
10. volatile long encoderVal; //编码器值
11. float velocity;
12. //自行定义其他全局变量
13.
14. //获取编码器值
15. void getEncoder(void)
16. {
17.     //自行补充代码
18. }
19. //PID 控制器，初始值供参考，Kp=1, Ti=10, Td=0, T=PERIOD
20. int pidController(float targetVelocity, float currentVelocity)
21. {
22.     float output;
23.     //自行补充代码
24.     return (int)output;
25. }
26. //直流电机控制
27. void control(void)
28. {
29.     //注意，大家的电机是轮子每转动一圈输出 390 个脉冲，实验室三编码器读取使用边缘触发中断，则可以实现脉冲的上升和下降计数。所以在计算速度的时候应采用 390*2=780 来计算速度。以下 PERIOD 为测速周期，ms 为单位， encoderVal 与实验三的编码器读取代码相同变量
30.     velocity = (encoderVal / 780.0) * 3.1415 * 2.0 * (1000 / PERIOD);
31.     //思路：根据电机当前转速
32.     encoderVal = 0;
33.     int output = pidController(TARGET, velocity);
34.     //TARGET 为目标速度值，目前大家的电机基本上最快为每秒转 50 弧度，所以不要设置超过这个值
35.     //以下代码很重要，大家可能需要根据电机实际转动，调整 INL1 和 INL2 的值
36.     if (output > 0)
37.     {
38.         digitalWrite(INL1, LOW);
39.         digitalWrite(INL2, HIGH);
40.         analogWrite(PWML, output);
41.     }
```

```

42.  else
43.  {
44.      digitalWrite(INL1, HIGH);
45.      digitalWrite(INL2, LOW);
46.      analogWrite(PWML, abs(output));
47.  }
48. }
49. void setup()
50. {
51.     //9,10 两个管脚的 PWM 由定时器 TIMER1 产生，这句程序改变 PWM 的频率，勿删
52.     TCCR1B = TCCR1B & B11111000 | B00000001;
53.     //这里自行添加代码，初始化各管脚，各全局变量，中断，串口通讯等
54.     MsTimer2::set(PERIOD, control);
55.     MsTimer2::start();
56. }
57. void loop()
58. {
59.     //撰写代码，使其能够通过“串口绘图器”观察速度波形，以下代码仅供参考
60.     Serial.print(velocity);
61.     Serial.print("\r\n");
62. }

```

附录：

参考程序（需要读懂程序，根据运行效果，调节其中的参数或修改某些语句）

```

1.  #include <MsTimer2.h>           //定时器库的 头文件
2.  //定义电机编码器 A B 相引脚
3.  #define ENCODER_A 2
4.  #define ENCODER_B 4
5.  //定义电机驱动的控制信号
6.  #define PWML 10
7.  #define INL1 12
8.  #define INL2 11
9.  #define PERIOD 20
10. #define TARGET -20
11. //全局变量
12. volatile long encoderVal; //编码器值
13. float velocity;
14. float u = 0;
15. float eI;
16. float eII;
17. float eIII;
18.
19. //获取编码器值

```

```

20. void getEncoder(void)
21. {
22.     if (digitalRead(ENCODER_A) == LOW)
23.     {
24.         if (digitalRead(ENCODER_B) == LOW)
25.         {
26.             encoderVal--;
27.         }
28.         else
29.         {
30.             encoderVal++;
31.         }
32.     }
33.     else
34.     {
35.         if (digitalRead(ENCODER_B) == LOW)
36.         {
37.             encoderVal++;
38.         }
39.         else
40.         {
41.             encoderVal--;
42.         }
43.     }
44. }
45. //PID 控制器, 初始值供参考, Kp=500, Ti=10, Td=400, T=PERIOD
46. int pidController(float targetVelocity, float currentVelocity)
47. {
48.     float output;
49.     eI = targetVelocity - currentVelocity;
50.     float Kp = 5, Ti = 140, Td = 80, T = PERIOD;
51.     float q0 = Kp * (1 + T / Ti + Td / T);
52.     float q1 = -Kp * (1 + 2 * Td / T);
53.     float q2 = Kp * Td / T;
54.     u = u + q0 * eI + q1 * eII + q2 * eIII;
55.     eIII = eII;
56.     eII = eI;
57.     if (u >= 255)
58.     {
59.         u = 255;
60.     }
61.     if (u <= -255)
62.     {
63.         u = -255;
64.     }

```



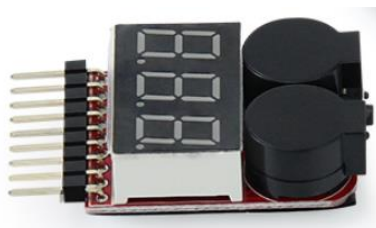
```

65.  output = u;
66.  return (int)output;
67. }
68. //直流电机控制
69. void control(void)
70. {
71.
72.  encoderVal = 0;
73.  int output = pidController(TARGET, velocity);
74.  //TARGET 为目标速度值，目前大家的电机基本上最快为每秒转 50 弧度，所以不要设置超过这个值
75.  //以下代码很重要，大家可能需要根据电机实际转动，调整 INL1 和 INL2 的值
76.  if (output > 0)
77.  {
78.      digitalWrite(INL1, LOW);
79.      digitalWrite(INL2, HIGH);
80.      analogWrite(PWML, output);
81.  }
82.  else
83.  {
84.      digitalWrite(INL1, HIGH);
85.      digitalWrite(INL2, LOW);
86.      analogWrite(PWML, abs(output));
87.  }
88. }
89. void setup()
90. {
91.  //9,10 两个管脚的 PWM 由定时器 TIMER1 产生，这句程序改变 PWM 的频率，勿删
92.  TCCR1B = TCCR1B & B11111000 | B00000001;
93.  //初始化各管脚，各全局变量，中断，串口通讯等
94.  pinMode(PWML, OUTPUT);
95.  pinMode(INL2, OUTPUT);
96.  pinMode(INL1, OUTPUT);
97.  pinMode(ENCODER_A, INPUT);
98.  pinMode(ENCODER_B, INPUT);
99.  attachInterrupt(0, getEncoder, CHANGE);
100.  Serial.begin(9600);
101.  MsTimer2::set(PERIOD, control);
102.  MsTimer2::start();
103. }
104. void loop()
105. {
106.  //撰写代码，使其能够通过“串口绘图器”观察速度波形
107.  Serial.print(velocity);
108.  Serial.print("\r\n");
109. }

```


第七章 轮式机器人自主巡线控制实验

重要：绝对不可以对锂电池过放电，当电池过放时，会永久损坏电池，当电压低于一定的压值时，必须停止使用电池，进行充电：2S 航模锂电池最低检测电压为 7.4V，3S 航模锂电池最低检测电压为 9.6V。使用过程，应时不时测量电池电压，或者全程接入电池报警器。



一、实验目的

- 1、掌握直流电机调速的方法；
- 2、理解 PID 调速原理及方法；
- 3、掌握电机位置闭环控制的实现原理；
- 4、了解轮式机器人巡线控制的硬件系统构成和软件实现方法；
- 5、掌握轮式机器人控制中电机速度环和位置环实现原理。

二、实验设备

智创-HIT-I 型实验箱，示波器，直流电源，电机转速测速计。

三、实验原理

1、基本原理

图 7.1 给出了循迹小车控制原理框图，该图由一个位置反馈闭环和两个速度反馈闭环构成，位置反馈体现在小车巡线运动的轨迹跟踪，位置反馈环节由多路寻迹光电传感器或者线性 CCD 摄像头完成；速度闭环体现在小车转弯时，左右车轮的速度改变，当小车运动到直线轨迹时，需要调节回原来的给定速度上。调节器采用 P、PI、PD 或 PID 算法，由 Arduino 控制器实现。

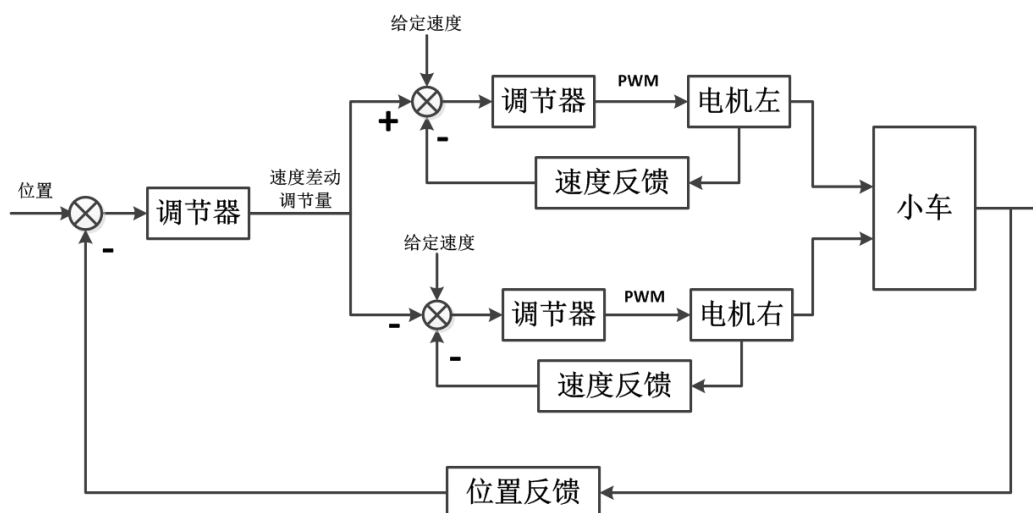


图 7.1 循迹小车控制原理框图

速度环使用增量式 PI 算法，给定的速度为人为设定，测量的速度由编码器测量获得，编码器有 A、B 两项，使用 M 测速法测量电机的速度（详见实验三）。位置环使用增量式 PD 算法，给定的位置信息设定为多路寻迹光电传感器或者 CCD 的中间阈值，测量的位置信息由多路寻迹光电传感器或者 CCD 实际测量获得，测量值与给定值的差为 PID 算法中的偏差 $e(k)$ 。

当使用 CCD 时，对循迹小车运行速度不能太快，因为速度太快，CCD 摄像头还没来得及处理下一位置的信息，小车将会跑出运行轨道，不可控制，图 7.2 给出小车运动轨道与运动速度简易分析图。如果使用多路寻迹光电传感器，则运行速度会比 CCD 更快。

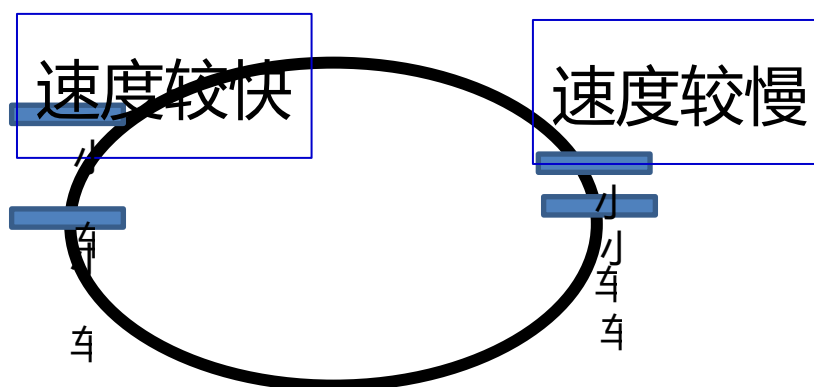


图 7.2 小车运行轨道与速度分析

2、光电寻迹模块的使用

在此我们使用寻迹模块 CTRT5000，该模块体积小，灵敏度较高，还可以通过转动上面的电位器来调节检测范围。



图 7.3 光电寻迹模块

- 1)、采用 TCRT5000 红外反射传感器
- 2)、检测距离：1mm~8mm 适用，焦点距离为 2.5mm
- 3)、比较器输出，信号干净，波形好，驱动能力强，超过 15mA。
- 4)、工作电压 3.3V-5V
- 5)、输出形式 OUT ：数字开关量输出（0 和 1）
- 6)、设有固定螺栓孔，方便安装

TCRT5000 传感器的红外发射二极管不断发射红外线，当发射出的红外线没有被反射回来或被反射回来但强度不够大时，光敏三极管一直处于关断状态，此时模块的输出端为低电平，指示二极管一直处于熄灭状态；被检测物体出现在检测范围内时，红外线被反射回来且强度足够大，光敏三极管饱和，此时模块的输出端为高电平，指示二极管被点亮。由于黑色具有较强的吸收能力，当循迹模块发射的红外线照射到黑线时，红外线将会被黑线吸收，导致循迹模块上光敏三极管处于关闭状态，此时模块上一个 LED 熄灭。在没有检测到黑线时，模块上 LED 常亮。模块在检测到黑线输出低电平，白线输出高电平。

循迹模块的工作一般要求距离待检测的黑线距离 1-2cm，因此我建议大家可以将循迹模块向下延伸。我自己是在硬纸板上面打了几个孔，固定循迹模块，每个模块之间可以留 1cm 左右的距离。安装时可以按照图 7.4 进行安装，装的越多，角度分辨率越高。实际上，寻线控制不需要太高的角度分辨率就可以运行很好。

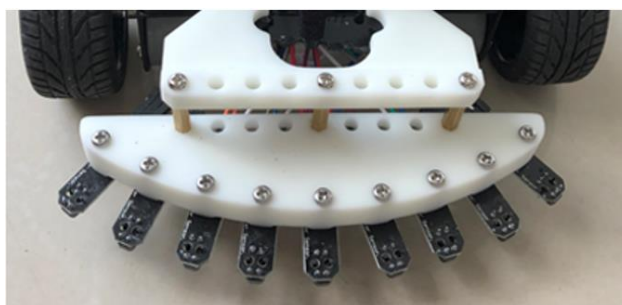


图 7.4 光电寻迹模块安装

四、实验步骤

（一）硬件系统构建步骤

1、系统电源供电：航模锂电池（最高 12.6V），5V 3.3V 电压模块（提供 5V 和 3.3V 电压），注意：系统中需要的电压，不能从 Arduino 控制器提供，必须由电压模块提供，因为 Arduino 控制器没有带负载能力，其输出功率有限。

2、电机位置环的搭建

线性 CCD 接线参考第五章实验模块三。

光电寻线传感器只需接上电源并将输出接入到 Arduino 的 I/O 口上即可。

3、电机速度环的搭建

使用电机驱动模块驱动左右轮直流电机，驱动模块可选用 L298N，TB6612 或自己设计的驱动电路。

4、电池电量检测电路

小车使用航模锂电池供电，当电池过放时，会永久损坏电池，当电压低于一定的压值时，必须停止使用电池，关闭电机：2S 航模锂电池最低检测电压为 7.4V，3S 航模锂电池最低检测电压为 9.6V，可以在程序里实现此功能。航模锂电池及充电器、接头等如图 7.3 所示。

注意：充电的时候不能使用电池，锂电池过放将导致电池永久损坏。3S 航模锂电池满电时为 12.6V，锂电池不能满电长时间（超过一周）存放，否则容易导致电池寿命下降，长期存放的最佳电压是 11.5V。

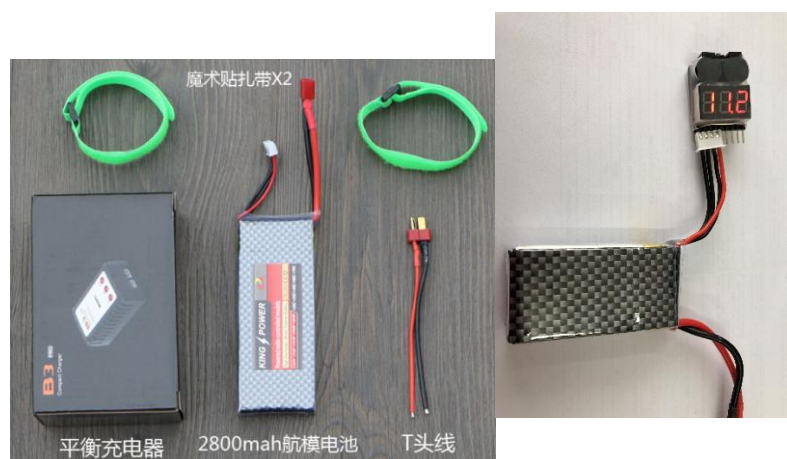


图 7.3 航模锂电池、充电器、电压检测

图 7.4 给出了精密电阻电压测量电路图，使用两个 1K 和 10K 的精密电阻对电池电压进行分压，OUT 接 Arduino 的 A0 引脚，A0 引脚可以作为 A/D 转换功能来使用，注意：Arduino 的 A/D 精度是 10 位，输入的最大电压为 5V，所以对输入的电压一定要限制在 0~5V 范围内。

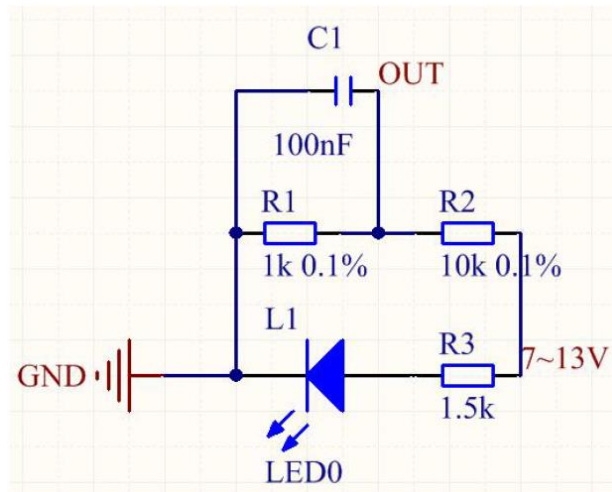


图 7.4 精密电阻电压测量电路图

A/D 计算方法（程序中的实现方法）：

采集出来的 AD 值和是 200 次的和 那么平均值： $AD = \text{sum}/200$

$$AD/1024 = V/5$$

$$\text{电压 } V = (AD * 5) / 1024$$

电压比是 10:1

$$\text{真实电压是 } \text{batt} = V * 11 = (AD * 5 * 11) / 1024$$

转化为以 mv 单位就是 $\text{batt} = AD * 5 * 11 * 1000 / 1024 = AD * 53.71$ 因为 $AD = \text{sum}/200$

所以真实电压就是 $\text{batt} = \text{sum} * 53.71 / 200 = \text{sum} * 0.5371 / 2$

注意程序中写的是 0.05371 是缩小了 10 倍。

图 7.5 为组装完成后的小车。

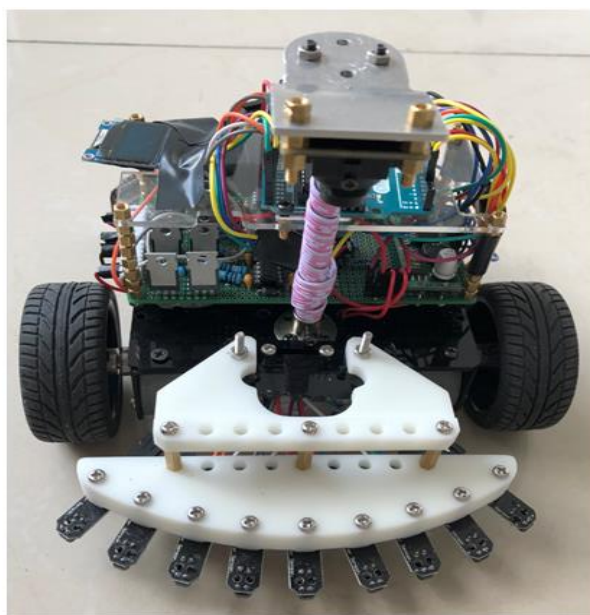


图 7.5 组装完成后的小车

（二）软件需要完成以下功能：

- ①打开小车电源开关，按下按键，则小车按照实际轨迹运行；
- ②电池电量检测，当电池电压低于 9.6V，则需停止小车运动，立即充电；
- ③ CCD 信号检测（A/D），并对获取的信号进行处理；或者检测光电寻迹传感器阵列
- ④电机控制（参考程序使用 298 驱动）：

A、速度环由编码器反馈构成（体现在转弯调速，左右轮速度改变，到直线运行时，需要将小车速度调回到原来给定的速度上，参考程序中控制方法采用 PI 控制实现速度环控制）；

B、位置环由传感器（CCD 或光电寻迹模块）反馈构成（体现在传感器采集数据即小车实际运行轨迹与标准值之间的偏差调节，参考程序中控制方法采用 PD 控制）。

（三）软件流程框图与涉及到的重要函数模块（图 7.6）

1、检测电池电量

BatteryVoltage()

2、获取 ccd ad 值

void CCD_ReadTsl1401()

计算 ccd 中值

uint8_t CCD_FindMiddleVal()

ccd 采样和计算转弯角度

void CCD_Run(void)

3、电机引脚初始化

void MotorInit(void)

4、小车运动数学模型，给定速度和转角

void KinematicAnalysis()

5、增量 PI 控制器

Incremental_PI_A()

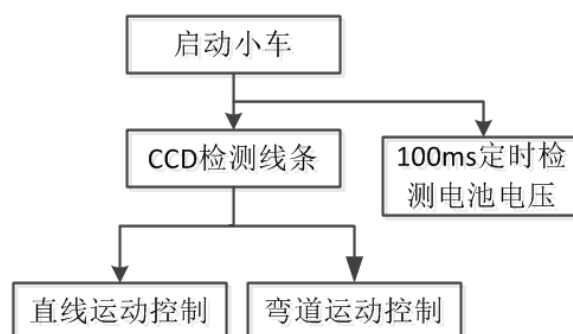


图 7.6 软件流程框图

四、实验参考程序设计

4.1 使用线性 CCD 循迹小车程序

(注：需要读懂程序，调节参数或部分语句)

```
1. #include <PinChangeInt.h>    //外部中断
2. #include <MsTimer2.h>        //定时中断
3.
4. //速度 PWM 设置的 KP 参数
5. #define VELOCITY_KP    0.5
6. //速度 PWM 设置的 PID KI 参数
7. #define VELOCITY_KI    0.5
8.
9. ////////////L298N 电机驱动引脚/////
10. #define AIN1    11
11. #define AIN2    5
12. #define BIN1    6
13. #define BIN2    3
14.
15. //编码器采集引脚
16. #define ENCODER_LEFT_A    8    //编码器采集引脚每路 2 个共 4 个
17. #define ENCODER_LEFT_B    4
18. #define ENCODER_RIGHT_A    7
19. #define ENCODER_RIGHT_B    2
20.
21. //CCD 引脚定义
22. #define CCD_DATA_PIN    A1 //15 //AO
23. #define CCD_CLK_PIN    A2 //16 //CLK
24. #define CCD_SI_PIN    A3 //17 //SI
25.
26. //按键引脚
27. #define KEY    A4 //18
28.
29. //电池电压检测引脚
30. #define BATT_VOL_PIN    A0 //13
31.
32. //左右编码器值(中间值获取后清零)
33. uint32_t encoderLeftVal;
34. uint32_t encoderRightVal;
35.
36. //小车速度
37. int velocityLeftEncoder; //后续做运算要用的左编码器值
38. int velocityRightEncoder; //后续做运算要用的右编码器值
39. int targetVelocity; //给定速度值
40. int turnAngle;    //转弯角度
41.
42. float leftVelocity; //给定速度和转弯角度计算得出的速度(左)
```

```

43. float rightVelocity;//给定速度和转弯角度计算得出的速度(右)
44.
45. //ccd 相关变量
46. uint8_t _adVal[128] = {0}; //保存 ccd 采样 ad 值
47. uint8_t _ccdThreshold; //ccd 阈值
48. uint8_t _ccdMiddleVal; //ccd 中值
49.
50. uint8_t carRunFlag = 0; //小车是否运行 1 运行 0 关闭电池电压低于 9V 时需关闭
51.
52. //电池电压
53. int batteryVoltage;
54.
55. //ccd 延时
56. void CCD_Delay(uint32_t tick)
57. {
58.     for (uint8_t i = 0; i < tick; i++);
59. }
60.
61. //ccd 初始化 IO 口
62. void CCD_Init(void)
63. {
64.     pinMode(CCD_CLK_PIN, OUTPUT);
65.     pinMode(CCD_SI_PIN, OUTPUT);
66. }
67.
68. //获取 ccd ad 值
69. void CCD_ReadTs11401(void)
70. {
71.     unsigned char i = 0;
72.     digitalWrite(CCD_CLK_PIN, HIGH);
73.     digitalWrite(CCD_SI_PIN, LOW);
74.     CCD_Delay(10);
75.     digitalWrite(CCD_SI_PIN, HIGH);
76.     digitalWrite(CCD_CLK_PIN, LOW);
77.     CCD_Delay(10);
78.     digitalWrite(CCD_CLK_PIN, HIGH);
79.     digitalWrite(CCD_SI_PIN, LOW);
80.     CCD_Delay(10);
81.
82.     for (i = 0; i < 128; i++) {
83.         digitalWrite(CCD_CLK_PIN, LOW);
84.         CCD_Delay(10); //调节曝光时间
85.         CCD_Delay(10); //调节曝光时间
86.         _adVal[i] = analogRead(CCD_DATA_PIN) >> 2;
87.         digitalWrite(CCD_CLK_PIN, HIGH);

```

```

88.     CCD_Delay(10);
89. }
90. }
91.
92. //计算 ccd 中值
93. uint8_t CCD_FindMiddleVal(void)
94. {
95.     unsigned int i, j, left, right, maxVal, minVal, middleVal;
96.
97.     CCD_ReadTsl1401();
98.
99.     maxVal = _adVal[0]; //动态阈值算法，读取最大和最小值
100.
101.     for (i = 5; i < 123; i++) //两边各去掉 5 个点
102.     {
103.         if (maxVal <= _adVal[i]) {
104.             maxVal = _adVal[i];
105.         }
106.     }
107.     minVal = _adVal[0]; //最小值
108.     for (i = 5; i < 123; i++)
109.     {
110.         if (minVal >= _adVal[i]) {
111.             minVal = _adVal[i];
112.         }
113.     }
114.
115.     _ccdThreshold = (maxVal + minVal) / 2; //计算出本次中线提取的阈值
116.
117.     for (i = 5; i < 118; i++) //寻找左边跳变沿
118.     {
119.         if (_adVal[i] > _ccdThreshold &&
120.             _adVal[i + 1] > _ccdThreshold &&
121.             _adVal[i + 2] > _ccdThreshold &&
122.             _adVal[i + 3] < _ccdThreshold &&
123.             _adVal[i + 4] < _ccdThreshold &&
124.             _adVal[i + 5] < _ccdThreshold)
125.         {
126.             left = i;
127.             break;
128.         }
129.     }
130.
131.     for (j = 118; j > 5; j--) //寻找右边跳变沿
132.     {

```

```

133.     if (_adVal[j] < _ccdThreshold &&
134.         _adVal[j + 1] < _ccdThreshold &&
135.         _adVal[j + 2] < _ccdThreshold &&
136.         _adVal[j + 3] > _ccdThreshold &&
137.         _adVal[j + 4] > _ccdThreshold &&
138.         _adVal[j + 5] > _ccdThreshold)
139.     {
140.         right = j;
141.         break;
142.     }
143. }
144.
145. middleVal = (right + left) / 2; //计算中线位置
146. return middleVal;
147. }
148.
149. //左编码器 A 相计数
150. void EncoderGetLeftA(void)
151. {
152.     encoderLeftVal++;
153. }
154.
155. //左编码器 B 相计数
156. void EncoderGetLeftB(void)
157. {
158.     encoderLeftVal++;
159. }
160.
161. //右编码器 A 相计数
162. void EncoderGetRightA(void)
163. {
164.     encoderRightVal++;
165. }
166.
167. //右编码器 B 相计数
168. void EncoderGetRightB(void)
169. {
170.     encoderRightVal++;
171. }
172.
173. //编码器引脚初始化
174. void EncoderInit(void)
175. {
176.     pinMode(ENCODER_LEFT_A, INPUT);
177.     pinMode(ENCODER_LEFT_B, INPUT);

```

```

178.  pinMode(ENCODER_RIGHT_A, INPUT);
179.  pinMode(ENCODER_RIGHT_B, INPUT);
180.  attachPinChangeInterrupt(ENCODER_LEFT_A, EncoderGetLeftA, CHANGE); //外部中断库函
    数
181.  attachPinChangeInterrupt(ENCODER_LEFT_B, EncoderGetLeftB, CHANGE);
182.  attachPinChangeInterrupt(ENCODER_RIGHT_A, EncoderGetRightA, CHANGE);
183.  attachPinChangeInterrupt(ENCODER_RIGHT_B, EncoderGetRightB, CHANGE);
184. }
185.
186.
187. /*****
188.  函数功能：小车运动数学模型
189.  入口参数：给定速度和转角
190.  *****/
191. void KinematicAnalysis(float velocity, float turn) {
192.     leftVelocity = velocity + turn;
193.     rightVelocity = velocity - turn;    //后轮差速
194. }
195.
196. /*****
197.  函数功能：增量 PI 控制器
198.  入口参数：编码器测量值，目标速度
199.  返回值：电机 PWM
200.  根据增量式离散 PID 公式
201.   $pwm += Kp[e(k) - e(k-1)] + Ki * e(k) + Kd[e(k) - 2e(k-1) + e(k-2)]$ 
202.   $e(k)$ 代表本次偏差
203.   $e(k-1)$ 代表上一次的偏差以此类推
204.  pwm 代表增量输出
205.  在我们的速度控制闭环系统里面，只使用 PI 控制
206.   $pwm += Kp[e(k) - e(k-1)] + Ki * e(k)$ 
207.  *****/
208. int Incremental_PI_A (int encoder, int target)
209. {
210.     static float bias, pwm, last_bias;
211.     bias = encoder - target;    //计算偏差
212.     pwm += VELOCITY_KP * (bias - last_bias) + VELOCITY_KI * bias; //增量式 PI 控制器
213.     if (pwm > 255) pwm = 255;    //限幅
214.     if (pwm < -255) pwm = -255;    //限幅
215.     last_bias = bias;    //保存上一次偏差
216.     return pwm;    //增量输出
217. }
218.
219. int Incremental_PI_B (int encoder, int target)
220. {
221.     static float bias, pwm, last_bias;

```

```

222.   bias = encoder - target;                                //计算偏差
223.   pwm += VELOCITY_KP * (bias - last_bias) + VELOCITY_KI * bias; //增量式 PI 控制器
224.   if (pwm > 255) pwm = 255;                                //限幅
225.   if (pwm < -255) pwm = -255;                              //限幅
226.   last_bias = bias;                                         //保存上一次偏差
227.   return pwm;                                              //增量输出
228. }
229.
230. /*****
231.   函数功能：赋值给 PWM 寄存器
232.   入口参数：PWM
233. *****/
234. void SetMotorPwm(int motorLeft, int motorRight) {
235.   if (motorLeft > 0)      analogWrite(AIN2, motorLeft), digitalWrite(AIN1, LOW); //
    //赋值给 PWM 寄存器
236.   else                  digitalWrite(AIN1, HIGH), analogWrite(AIN2, 255 + motorLeft)
    ; //赋值给 PWM 寄存器
237.
238.   if (motorRight > 0)     digitalWrite(BIN2, LOW), analogWrite(BIN1, motorRight)
    ; //赋值给 PWM 寄存器
239.   else                  analogWrite(BIN1, 255 + motorRight), digitalWrite(BIN2, HIGH); //赋值给 PWM 寄存器
240. }
241.
242. //小车控制 10ms 运行一次(定时器定时 10ms)
243. void CarControl(void)
244. {
245.   int motorLeftPwm, motorRightPwm;
246.
247.   //读取左右轮编码器数据，并清零，这就是通过 M 法测速（单位时间内的脉冲数）得到速度。
248.   velocityLeftEncoder = encoderLeftVal;
249.   velocityRightEncoder = encoderRightVal;
250.   encoderLeftVal = 0;
251.   encoderRightVal = 0;
252.   CCD_Run();
253.
254.   KinematicAnalysis(targetVelocity, turnAngle);
255.   motorLeftPwm = Incremental_PI_A(leftVelocity, velocityLeftEncoder);
256.   motorRightPwm = Incremental_PI_B(rightVelocity, velocityRightEncoder);
257.   if (carRunFlag)
258.   {
259.     SetMotorPwm(motorLeftPwm, motorRightPwm);
260.   }
261.
262.   // BatteryVoltage(); //检测电池电量

```

```

263.
264.  if (ButtonClick()) //按键开关运行
265.  {
266.      if (carRunFlag == 0)
267.      {
268.          carRunFlag = 1;
269.      }
270.      else
271.      {
272.          carRunFlag = 0;
273.          TurnOffMotor();
274.      }
275.  }
276. }
277.
278. //电机引脚初始化
279. void MotorInit(void)
280. {
281.     pinMode(AIN1, OUTPUT);
282.     pinMode(AIN2, OUTPUT);
283.     pinMode(BIN1, OUTPUT);
284.     pinMode(BIN2, OUTPUT);
285. }
286.
287. //ccd 采样和计算转弯角度
288. void CCD_Run(void)
289. {
290.     static float ccd_last_bias;
291.     float bias;
292.
293.     _ccdMiddleVal = CCD_FindMiddleVal();
294.     bias = _ccdMiddleVal - 64; //提取偏差
295.     turnAngle = bias * 0.12 + (bias - ccd_last_bias) * 3; //PD 控制
296.     ccd_last_bias = bias; //保存上一次的偏差
297.     if (turnAngle < -35)turnAngle = -35;//舵机角度限制
298.     if (turnAngle > 35)turnAngle = 35;//舵机角度限制
299. }
300.
301. //检测电池电压
302. void BatteryVoltage(void)
303. {
304.     static uint8_t count;
305.     static uint32_t sum;
306.     sum += analogRead(BATT_VOL_PIN); //累加电池 ad 值
307.     if (++count >= 200) //累计 200 次

```

```

308. {
309.     batteryVoltage = sum * 0.05371 / 2; //将电池电压降为 1/11 之后读取
310.     count = 0;
311.     sum = 0;
312.     Serial.println(batteryVoltage);
313.     if (batteryVoltage < 700)
314.     {
315.         carRunFlag = 0;
316.         TurnOffMotor();
317.     }
318. }
319. }
320.
321. /*****
322.     函数功能：按键扫描
323.     入口参数：无
324.     返回值：按键状态 0：无动作 1：单击
325. *****/
326. unsigned char ButtonClick (void)
327. {
328.     static byte flag_key = 1; //按键按松开标志
329.     if (flag_key && (digitalRead(KEY) == 0))
330.     { //如果发生单击事件
331.         flag_key = 0;
332.         if (digitalRead(KEY) == 0)
333.             return 1;
334.     }
335.     else if (digitalRead(KEY) == 1)
336.     {
337.         flag_key = 1;
338.     }
339.     return 0; //无按键按下
340. }
341.
342. //关闭电机
343. unsigned char TurnOffMotor(void)
344. {
345.     digitalWrite(AIN1, LOW); //电机驱动的电平控制
346.     digitalWrite(AIN2, LOW); //电机驱动的电平控制
347.     digitalWrite(BIN1, LOW); //电机驱动的电平控制
348.     digitalWrite(BIN2, LOW); //电机驱动的电平控制
349. }
350.
351. void setup()
352. {

```



```

353.   targetVelocity = 20; //设定目标速度
354.   pinMode(KEY, INPUT);
355.   CCD_Init();
356.   EncoderInit();
357.   MotorInit();
358.   MsTimer2::set(10, CarControl);    //使用 Timer2 设置 10ms 定时中断
359.   MsTimer2::start();                //定时器使能
360.   Serial.begin(9600);
361. }
362.
363. void loop()
364. {
365.   BatteryVoltage(); //检测电池电量
366.   delay(100);
367. }

```

4.2 使用光电寻迹模块的循迹小车程序

```

1. #include<MsTimer2.h> //定时器库的头文件
2. //-----定义管脚-----
3. #define ENCODER_A1 2 //电机 1
4. #define ENCODER_B1 5
5. #define ENCODER_A2 3 //电机 2
6. #define ENCODER_B2 4
7. #define PWM1 9
8. #define PWM2 10
9. #define L1 14 //左红外
10. #define L2 15
11. #define L3 16
12. #define R1 17 //右红外
13. #define R2 18
14. #define R3 19
15. #define M 13 //中红外
16.
17. //-----定义常值-----
18. #define PERIOD 20
19. #define Kp 7.0
20. #define Ti 50.0
21. #define Td 10.0
22.
23. //-----全局变量-----
24. float target1 = 2.0, t1; //左 保守
25. float target2 = 2.0, t2; //右 速度
26. volatile long encoderVal1; //编码器 1 值

```

```

27. float velocity1;           //转速 1
28. volatile long encoderVal2; //编码器 2 值
29. float velocity2;           //转速 2
30. float T = PERIOD;
31. float q0 = Kp * (1 + T / Ti + Td / T);
32. float q1 = -Kp * (1 + 2 * Td / T);
33. float q2 = Kp * Td / T;
34. float u1, ek11, ek12;
35. float u2, ek21, ek22;
36.
37. //-----测速与转向-----
    -
38. #define V 6.0 //基础速度
39. void control(void)
40. {
41.     if (digitalRead(M) == LOW)
42.     {
43.         target1 = V;
44.         target2 = V;
45.     }
46.     if (digitalRead(L1) == LOW) //低右转
47.     {
48.         target1 = V * 0.9;
49.         target2 = V * 0.5;
50.     }
51.     if (digitalRead(R1) == LOW) //低左转
52.     {
53.         target1 = V * 0.5;
54.         target2 = V * 0.9;
55.     }
56.     if (digitalRead(L2) == LOW) //中右转
57.     {
58.         target1 = V * 0.75;
59.         target2 = V * 0.15;
60.     }
61.     if (digitalRead(R2) == LOW) //中左转
62.     {
63.         target1 = V * 0.15;
64.         target2 = V * 0.75;
65.     }
66.     if (digitalRead(L3) == LOW) //直角右转
67.     {
68.         target1 = 0.65 * V;
69.         target2 = 0;
70.     }

```

```

71.  if (digitalRead(R3) == LOW)  //直角左转
72.  {
73.      target1 = 0;
74.      target2 = 0.65 * V;
75.  }
76.  target1 = target1 * 0.6 + t1 * 0.4;
77.  target2 = target2 * 0.6 + t2 * 0.4;
78.
79.  velocity1 = (encoderVal1 / 780.0) * 3.1415 * 2.0 * (1000 / PERIOD);
80.  encoderVal1 = 0;
81.  velocity2 = (encoderVal2 / 780.0) * 3.1415 * 2.0 * (1000 / PERIOD);
82.  encoderVal2 = 0;
83.
84.  int output1 = pidController1(target1, velocity1);
85.  analogWrite(PWM1, abs(output1));
86.  int output2 = pidController2(-target2, velocity2);
87.  if (output2 < 0)
88.      analogWrite(PWM2, abs(output2));
89.  t1 = target1;
90.  t2 = target2;
91. }
92.
93. //-----主函数-----
-
94. void setup() {
95.     //9,10 两个管脚的 PWM 由定时器 TIMER1 产生，这句程序改变 PWM 的频率，勿删
96.     TCCR1B = TCCR1B & B11111000 | B00000001;
97.     MsTimer2::set(PERIOD, control);
98.     MsTimer2::start();
99.     pinMode(ENCODER_A1, INPUT);
100.    pinMode(ENCODER_B1, INPUT);
101.    pinMode(ENCODER_A2, INPUT);
102.    pinMode(ENCODER_B2, INPUT);
103.    attachInterrupt(0, getEncoder1, CHANGE);
104.    attachInterrupt(1, getEncoder2, CHANGE);
105.    Serial.begin(9600);
106.    pinMode(PWM1, OUTPUT);
107.    pinMode(PWM2, OUTPUT);
108.    pinMode(R1, INPUT);
109.    pinMode(R2, INPUT);
110.    pinMode(R3, INPUT);
111.    pinMode(L1, INPUT);
112.    pinMode(L2, INPUT);
113.    pinMode(L3, INPUT);
114.    pinMode(M, INPUT);

```

```

115. }
116. void loop() {
117.
118.   Serial.print(velocity1); Serial.print("\t");
119.   Serial.println(velocity2);
120.   //Serial.print("\n");
121.   //Serial.println(u2);
122.   //Serial.println(u2);
123. }
124. //-----编码器中断函数-----
    ---
125. void getEncoder1(void)
126. {
127.   if (digitalRead(ENCODER_A1) == LOW)
128.   {
129.     if (digitalRead(ENCODER_B1) == LOW)
130.     {
131.       encoderVal1--;
132.     }
133.     else
134.     {
135.       encoderVal1++;
136.     }
137.   }
138.   else
139.   {
140.     if (digitalRead(ENCODER_B1) == LOW)
141.     {
142.       encoderVal1++;
143.     }
144.     else
145.     {
146.       encoderVal1--;
147.     }
148.   }
149. }
150. void getEncoder2(void)
151. {
152.   if (digitalRead(ENCODER_A2) == LOW)
153.   {
154.     if (digitalRead(ENCODER_B2) == LOW)
155.     {
156.       encoderVal2--;
157.     }
158.     else

```

```

159.     {
160.         encoderVal2++;
161.     }
162. }
163. else
164. {
165.     if (digitalRead(ENCODER_B2) == LOW)
166.     {
167.         encoderVal2++;
168.     }
169.     else
170.     {
171.         encoderVal2--;
172.     }
173. }
174. }
175. //-----PID 控制器-----
    --
176. int pidController1(float targetVelocity, float currentVelocity)
177. {
178.     float ek10;
179.     ek10 = targetVelocity - currentVelocity;
180.     u1 = u1 + q0 * ek10 + q1 * ek11 + q2 * ek12;
181.     if (u1 > 255)
182.     {
183.         u1 = 255;
184.     }
185.     if (u1 < -255)
186.     {
187.         u1 = -255;
188.     }
189.     ek12 = ek11;
190.     ek11 = ek10;
191.     return (int)u1;
192. }
193. int pidController2(float targetVelocity, float currentVelocity)
194. {
195.     float ek20;
196.     ek20 = targetVelocity - currentVelocity;
197.     u2 = u2 + q0 * ek20 + q1 * ek21 + q2 * ek22;
198.     if (u2 > 255)
199.     {
200.         u2 = 255;
201.     }
202.     if (u2 < -255)

```

```
203. {  
204.     u2 = -255;  
205. }  
206. ek22 = ek21;  
207. ek21 = ek20;  
208. return (int)u2;  
209. }
```

五、实验要求

- 1、构建巡线小车控制硬件系统；
- 2、使用 Arduino 编程实现线性 CCD 位置数据的采集，根据室内光线亮、暗变化，用串口调试助手分析数据，获取适合的阈值和曝光时间；
- 3、测量电池电压检测电路的 OUT 输出，分析对应的模拟电压 A 与转换后的数字量 D 之间的关系，如有偏差，请计算补偿数值；
- 4、使用 Arduino 编程实现小车巡线控制，改变小车速度，分析不同速度下，小车跟踪轨迹运行的效果。

六、拓展实验

- 1、手持遥感控制小车运动；
- 2、不规则轨道下小车巡线控制；
- 3、两辆小车同步运行控制，保持一定的车距。

七、实验报告要求

- 1、按照自己的理解写出小车巡线控制实验实现过程，采用的方法，得到的结论；
- 2、分析 CCD 数据，确定阈值和曝光时间，可以给出数据表；
- 3、电池电压检测的 A、D 数值计算；
- 4、对实验过程中出现的问题进行总结，写出解决方法与结论。

附录：

第一届机器人竞速赛最佳技术奖欧阳俊源、刘浩文

调试经验总结

1、直角弯的实现：定义左转右转的标志 bool 变量。

A、红外：对左数 1-4 个红外的黑线值进行计数，有 3 个左右的黑色即为左转，左转标志赋为 true，否则为 false，右转同理为 4-7。

B、CCD：CCD 的读取本来就很慢，需要进行优化，否则使速度有瓶颈。并且只需要在 0-63 的几个等分附近几个连续的数进行判断就可以了(如对 11-15，31-35，51-55)如果这几个值都是小于阈值即为左转。

C、在周期控制函数中对标记变量进行判断是否为 true，是左转则左轮-15 速度，右轮 15 速度，然后延时 300 毫秒。

2、Arduino 的 IDE 有毒，很难 Debug，也不好调试。只能分模块调试好再整合以免从头写到尾后全是 bug 不好找。

3、两个电机调速时候需要查看绘图器，如何在绘图器中绘制两条曲线？

```
Serial.print(a);Serial.print(“,”);Serial.println(b);
```

4、线性 ccd 读不出中线或读出的中线不稳定，移动时绘图器中不连续。

可能问题及解决方法：

黑胶布贴着的白纸大一点，清空桌面以免移动 CCD 时看到纸张边缘导致以为 CCD 读出的时错的。

光线干扰（ccd 对光线要求较高）比如阴影什么的。

调整曝光时间。

CCD 读出的 128 个灰度值中其实在两端会慢慢衰减(即看同一张白纸但绘图器上两边的值会低于中间的值，且越来越低)，这会使得有时候两边会被视为黑线处。可以用一张白纸在光线较好处试验，用串口绘图器看，把读出来的数据分别乘上某个系数使得绘图器中的图形是一条直线，然后再进行中线提取效果更好。

也可以调整动态阈值系数。

去除两边的点，或者去除个别长度低于(或许是 1-3)的极低值。

5、CCD 提取中线函数放在周期控制函数中。

现象：CCD 单独实验时能提取到中线，但整合在调速程序中时 CCD 的中线提取出来有问题

解决方法：放在 loop 中(但红外模块不需要)。

6、电量不足（格外注意），电池损坏。

现象：供电不足时小车一般会抖动，行动突然变得和程序不同或反常。

解决方法，可以一直将电压检测模块一直与电池相连，或者每隔一会儿检测一

次，或者设置一个检测电量的函数。

7、程序设计流程（使用光电传感器循迹）